

UNIVERSIDAD GERARDO BARRIOS

FACULTAD DE CIENCIAS Y TECNOLOGÍA

INGENIERÍA EN SISTEMAS Y REDES INFORMÁTICAS (GRUPO A2)

ACTIVIDAD:
LABORATORIO 1 – AVANCE I

TEMA:
NAVIA

PRESENTADO POR:

HERNÁNDEZ VÁSQUEZ, ABDIEL ALFREDO	**SMSS077825**
AMAYA BERRIOS, RUTH ISABEL	**SMSS061025**
ROQUE DIAZ, WILLIAM DANIEL	**SMSS025025**

CÁTEDRA:
PROGRAMACIÓN COMPUTACIONAL III

CATEDRÁTICA:
ING. WILLIAN ALEXIS MONTES GIRÓN

FECHA DE ENTREGA:
JUEVES 12 DE NOVIEMBRE DEL 2026

SAN MIGUEL CENTRO **EL SALVADOR**

Índice

Introducción 3
Objetivos 3
Objetivo general 3
Objetivos específicos 3
Delimitación 4
Tecnologías 4
Python 4
Flask 5
HTML, CSS y JavaScript 5
Leaflet 5
OpenStreetMap 5
Verdict 5
NumPy 6
Numba y llvmlite 6
Shapely 6
PyProj 6
PyArrow y Parquet 6
RapidFuzz 6
Git, GitHub y Git LFS 6
Requerimientos 7
Funcionales 7
No funcionales 7

Introducción

La planificación de recorridos requiere considerar tanto la distancia como las condiciones del tráfico, ya que la ruta más corta no siempre es la más rápida. Además, estas condiciones pueden cambiar durante el trayecto, especialmente en desplazamientos largos o en horarios de alta demanda.

Este proyecto presenta Navia - Navegación Adaptativa Vial con Inteligencia Artificial, una aplicación orientada a la planificación de rutas vehiculares en El Salvador. Navia utiliza una red vial procesada y el modelo estadístico Verdict para estimar las condiciones del tráfico según el tramo, el día de la semana y la hora.

A partir de estas estimaciones, el sistema calcula recorridos considerando la hora prevista de paso por cada carretera y propone hasta tres opciones con diversidad geográfica siempre que resulten suficientemente eficientes. También incorpora un mapa para visualizar el tráfico estimado y controles para seleccionar la fecha y hora de salida.

El modelo utiliza datos sintéticos apoyados en referencias parciales de tráfico habitual. Por ello, sus resultados representan estimaciones y no información de tráfico en tiempo real.

Objetivos

Objetivo general

Desarrollar una aplicación de planificación de rutas vehiculares para El Salvador que utilice estimaciones de tráfico por día y hora para identificar recorridos con menor tiempo previsto de viaje.

Objetivos específicos

- Preparar una red vial dirigida que represente las conexiones, los sentidos de circulación y las restricciones de giro disponibles en los datos.
- Integrar el modelo Verdict para consultar velocidades y condiciones estimadas de tráfico por tramo, día de la semana y hora.
- Implementar un cálculo de rutas que considere las condiciones previstas a la hora de paso por cada tramo.
- Proponer hasta tres opciones de recorrido con diversidad geográfica, evitando alternativas excesivamente similares o ineficientes.
- Diseñar una interfaz que permita seleccionar el inicio, el destino y la fecha y hora de salida, así como consultar los recorridos y sus tiempos estimados.
- Incorporar una vista del tráfico estimado que permita distinguir condiciones fluidas, moderadas y de atasco.
- Ejecutar las predicciones y el cálculo de rutas en CPU, sin requerir una tarjeta gráfica dedicada.

Delimitación

El proyecto se limita a la planificación de rutas vehiculares dentro de la red de El Salvador incluida en los datos procesados. La cobertura y las conexiones disponibles dependen de los insumos geográficos utilizados; no se garantiza la representación de todas las carreteras ni de sus cambios más recientes.

Las estimaciones de tráfico se generan mediante Verdict a partir de datos sintéticos y referencias parciales de condiciones habituales. Se consideran variaciones por día de la semana y hora, junto con ajustes de calendario implementados en el modelo. El sistema no recibe reportes de tráfico en vivo ni detecta automáticamente accidentes, obras o fenómenos meteorológicos.

El cálculo prioriza el menor tiempo previsto de viaje e incorpora las condiciones estimadas durante el recorrido. Puede presentar entre una y tres rutas, dependiendo de la disponibilidad de alternativas suficientemente distintas y eficientes.

La aplicación funciona de forma local y está orientada a su ejecución en Windows mediante Python. El procesamiento del modelo y las rutas se realiza en CPU. Se requiere conexión a Internet para instalar las dependencias

Esta versión no incluye cuentas de usuario o seguimiento GPS continuo.

Tecnologías

Tecnología	Aplicación en el proyecto
Python	Implementación del servidor, carga del modelo y cálculo de rutas.
Flask	Comunicación entre la interfaz y el procesamiento mediante una API HTTP.
HTML y CSS	Estructura y presentación de la interfaz web.
JavaScript	Interacción con el mapa, envío de solicitudes y presentación de resultados.
Leaflet	Visualización del mapa, los recorridos y las condiciones estimadas del tráfico.
OpenStreetMap	Datos geográficos utilizados para preparar la red vial y representar el mapa base.
NumPy	Manejo de matrices y operaciones numéricas para las predicciones.
Numba y llvmlite	Compilación y aceleración del cálculo de rutas en CPU.
Shapely	Procesamiento de geometrías y asociación de puntos con tramos de carretera.
PyProj	Transformación de coordenadas geográficas y métricas.
PyArrow	Lectura de los datos viales almacenados en formato Parquet.
RapidFuzz	Búsqueda aproximada de nombres de calles.
Verdict	Modelo estadístico de perfiles semanales y ajustes condicionales para estimar el tráfico.
Git, GitHub y Git LFS	Control de versiones y distribución del código, los modelos y los datos viales.

Python

Python se utiliza como lenguaje principal porque permite integrar el procesamiento de datos, la ejecución del modelo y el cálculo de rutas dentro de un mismo entorno. Sus bibliotecas numéricas y geográficas facilitan trabajar con los perfiles de tráfico y la red vial sin implementar desde cero todas las operaciones necesarias.

Flask

Flask se ajusta al alcance de Navia, que necesita recibir solicitudes de la interfaz, ejecutar cálculos y devolver resultados. Permite implementar los servicios de búsqueda de calles, consulta de tráfico, selección de modelos y cálculo de rutas.

La comunicación mediante datos JSON mantiene separada la presentación visual de la lógica del sistema, esta separación facilita modificar la interfaz o mejorar el modelo sin tener que reconstruir toda la aplicación.

HTML, CSS y JavaScript

Estas tecnologías permiten ofrecer una interfaz que se utiliza desde un navegador evitando desarrollar una aplicación de escritorio específica para presentar el mapa y sus controles.

HTML organiza los elementos de la interfaz, CSS permite adaptar su presentación a distintos tamaños de ventana y JavaScript gestiona la interacción del usuario y las solicitudes al servidor.

Leaflet

Leaflet proporciona los componentes necesarios para representar el mapa, colocar marcadores y dibujar rutas y capas de tráfico. Su integración con JavaScript ayuda a convertir los resultados del servidor en elementos visuales con los que el usuario puede interactuar.

Lo elegimos porque sus funciones se ajustan a las necesidades cartográficas del proyecto y permiten mantener el cálculo de rutas en Python mientras la biblioteca se encarga de su representación y exploración en el navegador.

OpenStreetMap

OpenStreetMap aporta datos geográficos y atributos viales que permiten preparar la red utilizada por Navia. Esta información incluye elementos importantes para el recorrido vehicular, como conexiones entre vías, sentidos de circulación y restricciones de giro disponibles en los insumos.

Trabajar con una red procesada y almacenada localmente permite controlar cómo se representan sus conexiones y cómo se aplican los costes de tráfico durante la búsqueda, así el cálculo de rutas utiliza la red propia del proyecto,mientras el mapa base proporciona el contexto visual.

Verdict

Verdict se utiliza porque representa el comportamiento estimado del tráfico por tramo, día de la semana y hora. Este modelo permite consultar las condiciones de una carretera según el momento previsto de llegada a ella, lo que resulta necesario para calcular recorridos que atraviesan diferentes horarios.

El modelo puede ejecutarse en CPU y consultarse sin volver a entrenarlo durante cada búsqueda. Su formato también permite incorporar futuras versiones compatibles, favoreciendo la mejora del sistema a medida que se disponga de más observaciones. El uso de datos sintéticos permite desarrollar y evaluar el funcionamiento del proyecto ante la disponibilidad limitada de datos reales, reconociendo que sus resultados son aproximaciones.

NumPy

NumPy permite almacenar y procesar los valores del modelo mediante matrices numéricas. Esta estructura se adapta a los perfiles organizados por tramo, día y hora, y facilita calcular velocidades, factores de demora y tiempos de recorrido.

Las operaciones sobre conjuntos de valores permiten procesar múltiples tramos de forma conjunta. Esto resulta útil para generar las condiciones de tráfico de una zona y preparar los costes utilizados por el algoritmo de rutas.

Numba y llvmlite

Numba se utiliza para compilar las partes del cálculo que realizan numerosas operaciones repetitivas, especialmente durante la búsqueda de rutas y la integración de tiempos de recorrido. Lo utilizamos para aprovechar la CPU y reducir el tiempo dedicado a estos cálculos.

llvmlite proporciona la infraestructura de compilación utilizada por Numba. Ambos permiten acelerar el procesamiento sin introducir una dependencia de GPU o CUDA, en concordancia con el requisito de utilizar equipos sin tarjeta gráfica dedicada.

Shapely

Shapely facilita las operaciones con las geometrías de las carreteras, como medir distancias, localizar el punto más cercano sobre una vía y obtener las porciones de los tramos recorridos.

PyProj

PyProj permite transformar las coordenadas geográficas en coordenadas adecuadas para realizar cálculos métricos. Esto es necesario porque las posiciones del mapa se expresan en latitud y longitud mientras ciertas operaciones espaciales requieren distancias en metros.

PyArrow y Parquet

La red vial incluye información tabular y geométrica que debe recuperarse al iniciar la aplicación. Parquet permite conservar estos datos en un formato estructurado y comprimido mientras PyArrow proporciona las herramientas necesarias para leerlos.

RapidFuzz

RapidFuzz permite implementar búsquedas aproximadas de nombres de calles. Esta capacidad facilita encontrar resultados cuando el usuario escribe un nombre incompleto o presenta pequeñas diferencias respecto al nombre almacenado.

La búsqueda se realiza sobre los datos locales lo que permite integrarla con la selección de puntos del mapa sin depender de un servicio externo de geocodificación.

Git, GitHub y Git LFS

Git permite registrar cambios en el código y mantener versiones identificables del proyecto. GitHub facilita su distribución, la documentación y la publicación de versiones descargables.

Git LFS complementa estas herramientas al gestionar los archivos binarios grandes del modelo y la red vial.

Requerimientos

Funcionales

Código	Requerimiento
RF-01	El sistema deberá permitir seleccionar los puntos de inicio y destino sobre el mapa.
RF-02	El sistema deberá permitir buscar calles por nombre dentro de la red vial disponible.
RF-03	El sistema deberá permitir modificar los puntos seleccionados y reiniciar el recorrido.
RF-04	El sistema deberá permitir seleccionar la fecha y hora de salida.
RF-05	El sistema deberá calcular la ruta principal buscando el menor tiempo total previsto, incluyendo las demoras asociadas al tráfico.
RF-06	El sistema deberá consultar las condiciones previstas a la hora de paso por cada tramo y considerar los cambios de hora y día durante el recorrido.
RF-07	El sistema deberá aplicar los sentidos de circulación, accesos y restricciones de giro incorporados en la red procesada.
RF-08	El sistema deberá presentar hasta tres rutas con diversidad geográfica, mostrando menos opciones cuando las restantes sean demasiado similares o ineficientes.
RF-09	El sistema deberá mostrar la distancia, el tiempo previsto de viaje y la hora estimada de llegada de cada ruta.
RF-10	El sistema deberá permitir seleccionar una ruta para destacar su recorrido en el mapa.
RF-11	El sistema deberá permitir alternar entre una vista normal y una vista de tráfico estimado.
RF-12	La vista de tráfico deberá distinguir condiciones fluidas, moderadas y de atasco, y mostrar la velocidad estimada al seleccionar una carretera.
RF-13	El sistema deberá mostrar las calles locales al acercarse al mapa y ocultarlas en vistas más amplias.
RF-14	El sistema deberá informar cuando los puntos sean inválidos, no exista una conexión disponible o falten archivos necesarios para ejecutar las consultas.

No funcionales

Código	Requerimiento
RNF-01	Las predicciones y el cálculo de rutas deberán ejecutarse en CPU.
RNF-02	El sistema deberá reutilizar los datos y modelos cargados y limitar los cálculos simultáneos para controlar el consumo de recursos.
RNF-03	La interfaz deberá indicar cuándo una operación esté en curso y mantenerse disponible para la interacción del usuario.
RNF-04	La visualización deberá solicitar únicamente los datos de la zona visible y adaptar el detalle al nivel de acercamiento del mapa.
RNF-05	Los controles principales deberán permitir identificar con claridad el modelo, los puntos del recorrido y la fecha y hora seleccionados.
RNF-06	La interfaz deberá adaptarse a distintos tamaños de ventana, manteniendo accesibles sus controles principales.
RNF-07	La lógica de predicción, el cálculo de rutas y la interfaz deberán mantenerse separados para facilitar su mantenimiento.
RNF-08	El sistema deberá verificar la integridad y compatibilidad de los archivos del modelo antes de utilizarlos.
RNF-09	Las respuestas de solicitudes anteriores no deberán sustituir los resultados correspondientes a una selección más reciente del usuario.
RNF-10	El proyecto deberá incluir sus archivos de ejecución y una declaración de dependencias con versiones fijadas para facilitar su instalación.